Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Data Structures Practical

Practical - VI

Roll No. :S010	Name: ANJALI GUPTA
Class : SYIT	Batch : 1
Date of Assignment : 29/07/2026	Date/Time of Submission : 29/07/2026

PART – A : Thoery

A **Binary Tree** is a non-linear data structure in which each node has at most two children: a **left child** and a **right child**.

Tree Traversal is the process of visiting every node of a binary tree exactly once in a specific order. The three basic traversal methods are:

- **Pre-order Traversal (Root → Left → Right):** Visits the root node first, then the left subtree, and finally the right subtree.
- **In-order Traversal (Left → Root → Right):** Visits the left subtree first, then the root node, and finally the right subtree.
- **Post-order Traversal (Left → Right → Root):** Visits the left subtree first, then the right subtree, and finally the root node.

These traversal techniques are commonly used for searching, displaying, and processing the nodes of a binary tree. They are usually implemented using **recursion**.

PART – B : Question

Tree Traversal: Write a program to:

a. Implement pre-order,

CODE:

```
class Node:
     def __init__(self,item):
         self.left=None
         self.right=None
         self.val=item

def inorder(root):
    if root:
         inorder(root.left)
         print(str(root.val)+"->",end='')
         inorder(root.right)


def postorder(root):
    if root:
         postorder(root.left)
         postorder(root.right)
         print(str(root.val)+"->",end='')

def preorder(root):
    if root:
         print(str(root.val)+"->",end='')
         preorder(root.left)
         preorder(root.right)


root = Node(1)
root.left =Node(2)
root.right=Node(3)
root.left.left=Node(4)
root.left.right=Node(5)

print("Inorder Traversal")
inorder(root)

print("\npostorder Traversal")
postorder(root)
```

```
root = Node(1)
root.left =Node(2)
root.right=Node(3)
root.left.left=Node(4)
root.left.right=Node(5)

print("Inorder Traversal")
inorder(root)

print("\npostorder Traversal")
postorder(root)

print("\npreorder Traversal")
preorder(root)
```

OUTPUT:

```
Inorder Traversal
4->2->5->1->3->
postorder Traversal
4->5->2->3->1->
preorder Traversal
1->2->4->5->3->
```

b. in-order,
CODE:

File Edit Format Run Options Window Help

```
class Node:
     def __init__(self,item):
         self.left=None
         self.right=None
         self.val=item

def inorder(root):
    if root:
         inorder(root.left)
         print(str(root.val)+"->",end='')
         inorder(root.right)


def postorder(root):
    pass

def preorder(root):
    pass

root = Node(1)
root.left =Node(2)
root.right=Node(3)
root.left.left=Node(4)
root.left.right=Node(5)

print("Inorder Traversal")
inorder(root)
```

OUTPUT:

```
========================
Inorder Traversal
4->2->5->1->3->
```

c. Post-order traversal of a binary tree.

CODE:

```
class Node:
     def __init__(self,item):
         self.left=None
         self.right=None
         self.val=item

def inorder(root):
    if root:
         inorder(root.left)
         print(str(root.val)+"->",end='')
         inorder(root.right)


def postorder(root):
    if root:
         postorder(root.left)
         postorder(root.right)
         print(str(root.val)+"->",end='')

def preorder(root):
    if root:
         inorder(root.left)
         inorder(root.right)
         print(str(root.val)+"->",end='')

root = Node(1)
root.left =Node(2)
root.right=Node(3)
root.left.left=Node(4)
root.left.right=Node(5)

print("Inorder Traversal")
inorder(root)

print("\npostorder Traversal")
postorder(root)
```

OUTPUT:

```
=======================
Inorder Traversal
4->2->5->1->3->
postorder Traversal
4->5->2->3->1->
```

Curiosity Questions:

1. In a BST, which side (left or right) has **smaller numbers** than the root?
2. If we insert numbers 10, 20, 5 into a BST, which number will be at the root?
3. What will the inorder traversal of a BST give — sorted numbers or random numbers?
4. Where will the **largest number** in a BST always be found?
5. If the root of a BST is 50 and we search for 70, will we go left or right? Why?
6. If a BST has only **one node**, what is its inorder traversal?
7. Can we create a BST from characters like A, B, C instead of numbers?
8. If we insert numbers 5, 10, 15, 20 in order, will the tree be balanced or like a straight line?
9. In a BST, do we check the left child first or the right child first in inorder traversal?
10. If we delete the root of a BST, will the tree disappear?